RAG Model

Hanna Roed
hanna.roed@berkeley.edu

Arya Raeesi
aryaraeesi@berkeley.edu

Rohan Bijukumar
rohanbijukumar@berkeley.edu

Question 1: QA Data Creation

We built our QA dataset by crawling UC Berkeley EECS pages and extracting page text into a JSONL corpus. Our URL crawler was designed to discover and traverse links whose host includes `eecs.berkeley.edu`, while excluding non-HTML and unsupported file types. Using this corpus, we generated candidate factoid QA pairs with LLM assistance, focusing on answer types such as emails, phone numbers, dates, names, and program-related facts.

Each QA instance is stored in structured format with `question`, `answer`, and `url`. We constructed a blind inter-annotator agreement (IAA) subset of 30 questions (30% of the validation set). Inter-annotator agreement (IAA), measured on this subset, was 86.7% Exact Match and 92.4 token-level F1. We also exported question-only files for evaluation and created additional holdout sets to assess generalization.

The final validation dataset contains 100 questions from 33 unique URLs. Additional holdout sets contain 30, 31, and 31 questions (92 total). Average question length is 9.7 tokens (median 10), and average answer length is 2.19 tokens (median 2; range 1–7). Lastly, Table 1 gives 3 examples of QA pairs from our validation set.

Table 1: Representative QA pairs from our validation set.

Question	Answer	Source URL
What email should applicants use for graduate program questions?	`gradadmissions@eecs.berkeley.edu`	`eecs.berkeley.edu/contact`
Where is the Computer Science Division office located?	387 Soda Hall	`eecs.berkeley.edu/contact`
Who is the Director of External Relations?	Bennett Agnew	`eecs.berkeley.edu/people/staff/external-relations-staff`

Question 2: Retrieval Corpus

We constructed our retrieval corpus by crawling EECS-domain pages and storing extracted page text in JSONL format. Our primary corpus contains 8,417 documents, with an average document length of 292.9 words. The crawl includes the main EECS site and EECS-related subdomains. At retrieval time, we represent each chunk using enriched text that combines the page title, URL-derived lexical features (host/path tokens), and chunk content, which improves lexical matching for sparse retrieval.

To evaluate corpus quality, we report both end-to-end QA metrics and retrieval-specific diagnostics. On the validation set, the system achieves token-level F1 of 0.7579. The correct source URL appears within the top four retrieved results for 73% of questions, and the retrieved context contains the gold answer string for 81% of questions. These diagnostics help separate retrieval/corpus errors from generation errors.

We also compared corpus variants built from the same crawl pipeline. In our experiments, the filtered/raw crawl corpus (`crawl_eecs.jsonl`) performed slightly better than aggressively cleaned alternatives. A likely reason is that heavy cleaning removes lexical cues (exact strings, local formatting markers, and token patterns) that are useful for BM25-style matching.

Question 3: RAG System

Our RAG system uses a retrieval-first design with a language-model answerer and a deterministic fallback. The implementation is in ‘rag/model.py‘ and follows a fixed sequence: load corpus documents, split them into overlapping chunks, index chunk text with BM25 [1], retrieve top candidates, rerank with query-type heuristics, and then answer with an instruction-tuned LLM.

The retrieval stage is not plain BM25 over raw text. Each chunk is represented with enriched retrieval text that includes title terms, URL-host/path tokens, and canonical URL features (for example, indicators for legacy pages and PDF-like paths). For each question, the system also builds weighted query variants (for person, email, location, date/year, and related question types), fuses scores, and applies domain-specific reranking [2] bonuses/penalties.

In generation mode, the final prompt contains the question plus top-ranked context blocks, and the model returns a short-form answer under strict formatting instructions (e.g., return only answer text, use Yes/No for binary questions, output “unknown” when unsupported). If the LLM call fails or is disabled, the system switches to an extractive fallback that combines regex/entity detectors [3] with relation patterns and overlap-based candidate scoring.

However, all of this was simply not accurate enough. After the hidden dev set was released, we made significant improvements.

In the final system, we also include a lightweight question-prior module in addition to retrieval and generation. This module uses previously curated QA files in two ways: (1) optional exact normalized question lookup, and (2) soft transfer from similar

past questions using token-overlap/Jaccard similarity with question-type consistency. We also derive URL hints from similar questions and use them as a retrieval bias (not as a hard filter).

This design improved dev-set accuracy substantially, but it can overfit if evaluated on distributions similar to the prior QA files. For this reason, we treat prior-assisted performance and pure retrieval+generation performance as separate operating modes, and we report ablations that isolate the contribution of LLM, retrieval, and fallback extraction.

The final answer is postprocessed for consistency (course-code canonicalization, degree format normalization, date/year/number cleanup, and unknown normalization). This design choice is important because the grader is sensitive to short-answer formatting differences. Overall, the system intentionally mixes learned generation with hand-crafted retrieval/extraction controls to balance robustness and score optimization.

Question 4: Ablations

We ran controlled ablations to isolate where performance gains come from and where the pipeline is still fragile. The following lists formalize our evaluation metrics when performing our two ablations on the validation and holdout sets.

Ablation 1: LLM-enabled vs fallback-only

Validation set:

- LLM enabled: F1 = 0.7579, EM = 0.6500
- LLM disabled (‘–no-llm‘): F1 = 0.2844, EM = 0.1800

Holdout-mini set:

- LLM enabled: F1 = 0.9239, EM = 0.8333
- LLM disabled (‘–no-llm‘): F1 = 0.6496, EM = 0.6000

Interpretation: the largest single effect is the LLM layer. Retrieval/extraction alone is useful but much weaker, especially on validation where answer formatting and disambiguation are harder [4].

Ablation 2: Retrieval depth (top-k) in fallback-only mode

Validation fallback-only:

- top-k = 3: F1 = 0.2867
- top-k = 8: F1 = 0.2698

Holdout-mini fallback-only:

- top-k = 3: F1 = 0.6718
- top-k = 8: F1 = 0.6496

Interpretation: larger retrieval depth did not help in this setting and often hurt slightly, likely because extra chunks add distractor spans that reduce extraction precision [5].

Question 5: Error Analysis

We analyzed failures from the best validation run (‘predictions.txt‘) and focused on exact zero-score cases (F1 = 0.0). There were 20 such cases out of 100 questions.

The largest category was retrieval miss/no-answer behavior, where the system predicted ‘unknown‘ despite an answer existing in the corpus. A second common category was wrong-entity selection, where retrieval found related pages but the model chose a nearby incorrect person/email/entity.

We also observed a non-trivial set of false negatives caused by metric limitations rather than true semantic failure. Representative examples include ‘CS 70‘ vs ‘CS70‘, ‘At least three‘ vs ‘3‘, and spelling variants such as ‘canceled‘ vs ‘Cancelled‘. Under strict token-overlap F1, these are scored as complete failures even when meaning is equivalent [6].

Estimated error breakdown on the 20 zero-F1 cases:

- Retrieval miss / predicted unknown: 9
- Wrong entity from related context: 5
- False negatives due to metric limitations: 4
- Extraction/postprocessing artifact: 2

To improve evaluation quality, we would keep strict EM/F1 as the headline metric but add normalization-aware matching for common equivalences (alphanumeric spacing, number forms, minor spelling variants) and report a secondary semantic-equivalence metric for analysis.

Question 6: Takeaways and Future Ideas

The main takeaway from our experiments is that retrieval quality remains the core bottleneck. Validation diagnostics support this: when the correct URL or answer-bearing context is retrieved, downstream answer quality is much higher; when retrieval misses, the model frequently returns ‘unknown‘ or selects a plausible but incorrect entity.

A second takeaway is that robustness varies by page type and formatting style. The system is strongest on common modern EECS page layouts and weaker on legacy or structurally unusual content. This gap likely explains why performance can drop sharply on hidden or legacy-style evaluations.

With more time, we would prioritize three directions. First, improve retrieval coverage (especially for legacy/www2 and document-like pages) using hybrid sparse+dense retrieval and stronger reranking [7]. Second, strengthen fallback extraction so performance is less dependent on LLM behavior. Third, make evaluation more informative by tracking retrieval diagnostics (URL recall and answer-in-top-k) alongside EM/F1 on every experiment.

Contribution Statement

We contributed equally across the full project lifecycle, either by dividing the work or taking on emphasis in a certain phase of the project. Those phrases include but are not limited to: dataset and question/answer creation, text crawling, retrieval corpus creation, RAG model creation, experimenting, evaluation, and report writing.

GenAI Statement

We acknowledge the use of ChatGPT, CoPilot, and Google AI Overview's to help clarify error messages, answer conceptual questions about the course material, and debug syntax issues with unfamiliar packages. Additionally, while it's not GenAI in the practical sense, we looked online and took inspiration from very similar implementations (chunking, BM25 ranking, etc). In the end, we used Google's AI summary to help find lots of source material. Additionally, LLMs (like Google AI's summary) suggested we implement a batch mode to improve the efficiency of our evaluation. This was a significant performance improvement when testing our project, which might have been over a hundred times.

We used ChatGPT to help interpret error messages and to clarify syntax for our model.py file. Similarly, since our RAG architecture deviates from the lecture design, the LLM helped clarify conceptual misunderstandings. CoPilot was used to save time with tab-based completion, but we have the ability to explain and even independently replicate the work done in this document if asked by an instructor. Google's AI Overview helped summarize concepts when querying the internet for additional resources. Additionally, it helped us quickly find references and resources for us to model our RAG's architecture after. We additionally used ChatGPT to help debug issues regarding our report's formatting in LaTeX.

References

[1] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in IR*, 2009.

[2] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019.

[3] Pranav Rajpurkar et al. Squad: 100,000+ questions for machine comprehension of text. *EMNLP*, 2016.

[4] Tom Brown et al. Language models are few-shot learners. *NeurIPS*, 2020.

[5] Vladimir Karpukhin et al. Dense passage retrieval for open-domain qa. *EMNLP*, 2020.

[6] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *EMNLP*, 2019.

[7] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction. *SIGIR*, 2020.